

云南大学软件学院

实 验 报 告

课程：_____Web 技术及应用开发实践_____任课教师：_____陈清毅_____

姓名：_____张润泽_____学号：_____20231120231_____专业：_____网络空间安全_____日期：_____2025-12-10_____成绩：_____

实验 4-5 NodeJs及Serverless应用开发

一、实验目的

- (1) 学习基于 NodeJs 的现代 Web 开发模式
- (2) 学习通过后端框架和 Typescript 搭建一个数据库交互和 API 访问能力的后端服务。
- (3) 学习部署一个 Serverless 到云端

二、实验内容

- (1) 安装 NodeJS 及 NPM 环境配置。使用框架提供的 CLI 工具创建项目脚手架
- (2) 后端框架目前不要求 UI 内容。可以选用 nestJs、koaJs、Express 等
- (3) 设计/使用 一个数据库 (MySQL、SqlServer、MongoDB、Oracle 皆可)，具备 1, 2 个有数据的表
- (4) 设计与数据
- (5) 类，使用 TypeORM 一类的数据库访问框架，完成从 NodeJs 中进行数据 CURD 的操作。
- (6) 设计用于服务的 API (Restful、GRAPHQL) 的 Controller，具备基本的 Get 和 Post 方法。
- (7) 使用 Postman/ApiFox 编写测试用例，测试 API Controller。
- (8) 部署当前项目到云端 Serverless 服务，可以选用腾讯云或阿里云

三、实验要求

- (1) 实验完成实现为 3 周
- (2) 使用 VsCode 或者 WebStorm 搭建基本的 Web 应用开发环境, 新建
- (3) 完成实验内容，页面源码提交到码云，代码分支。要包含所有源代码文件。
- (4) JavaScript 中的关键部分要求有注释，注释量不低于 10%
- (5) 代码要求独立完成，不得抄袭网站或其他同学代码。
- (6) 电子报告同步提交到 GIT, 纸质实验报告按老师要求提交。

三、关键实验步骤（请粘贴关键步骤、代码、实验结果）

1. 项目初始化与环境配置

步骤 1: 创建 NestJS 项目

使用 NestJS CLI 创建项目

```
npx @nestjs/cli new nodejs-serverless-app
```

```
cd nodejs-serverless-app
```

步骤 2: 安装必要依赖

核心依赖

```
npm install @nestjs/mongoose @nestjs/config @nestjs/swagger
```

```
npm install mongoose class-validator class-transformer
```

```
npm install swagger-ui-express
```

开发依赖

```
npm install --save-dev @types/node ts-node nodemon typescript
```

步骤 3: 配置环境变量 (.env)

```
MONGODB_URI=mongodb://localhost:27017/nodejs-experiment
```

```
PORT=3000
```

```
JWT_SECRET=your-secret-key-here
```

```
JWT_EXPIRES_IN=7d
```

2. 数据库设计与模型定义

步骤 4: 用户数据模型 (user.schema.ts)

```
@Schema({ timestamps: true })
```

```
export class User {
```

```
  @Prop({ required: true, unique: true, trim: true })
```

```
  username: string;
```

```
  @Prop({ required: true, unique: true, trim: true, lowercase: true })
```

```
  email: string;
```

```
  @Prop({ required: true, minlength: 6 })
```

```
  password: string;
```

```
  @Prop({ enum: ['active', 'inactive'], default: 'active' })
```

```
  status: string;
```

```
  @Prop({ enum: ['user', 'admin'], default: 'user' })
```

```
  role: string;
```

```
}
```

步骤 5: 产品数据模型 (product.schema.ts)

```
@Schema({ timestamps: true })
```

```
export class Product {
```

```
  @Prop({ required: true, trim: true })
```

```
  name: string;
```

```
  @Prop({ required: true, min: 0 })
```

```
  price: number;
```

```
  @Prop({ required: true, min: 0, default: 0 })
```

```

    stock: number;

    @Prop({ required: true, enum: ['electronics', 'clothing', 'food', 'books',
'other'] })
    category: string;

    @Prop({ min: 0, max: 5, default: 0 })
    rating: number;

    @Prop({ min: 0, default: 0 })
    salesCount: number;
}

```

3. 数据验证与 DTO 设计

步骤 6: 创建用户 DTO (create-user.dto.ts)

```

export class CreateUserDto {
    @IsNotEmpty({ message: '用户名不能为空' })
    @MinLength(3, { message: '用户名至少需要 3 个字符' })
    @MaxLength(20, { message: '用户名不能超过 20 个字符' })
    username: string;

    @IsNotEmpty({ message: '邮箱不能为空' })
    @IsEmail({}, { message: '请输入有效的邮箱地址' })
    email: string;

    @IsNotEmpty({ message: '密码不能为空' })
    @MinLength(6, { message: '密码至少需要 6 个字符' })
    password: string;
}

```

4. 业务逻辑实现

步骤 7: 用户服务层 (user.service.ts)

```

async create(createUserDto: CreateUserDto): Promise<User> {
    // 检查用户名和邮箱是否已存在
    const existingUser = await this.userModel.findOne({
        $or: [
            { username: createUserDto.username },
            { email: createUserDto.email }
        ]
    }).exec();

    if (existingUser) {
        throw new ConflictException('用户名或邮箱已存在');
    }
}

```

```
// 创建新用户
const newUser = new this.userModel(createUserDto);
return newUser.save();
}
```

步骤 8: 产品服务层库存管理 (product.service.ts)

```
async increaseStock(id: string, quantity: number): Promise<Product> {
  const product = await this.productModel.findById(id).exec();
  if (!product) {
    throw new NotFoundException(`产品 ID ${id} 不存在`);
  }

  if (quantity <= 0) {
    throw new ConflictException('增加的数量必须大于 0');
  }

  return this.productModel
    .findByIdAndUpdate(id, { $inc: { stock: quantity } }, { new: true })
    .exec();
}
```

5. API 控制器实现

步骤 9: 用户控制器 (user.controller.ts)

```
@Post()
@ApiOperation({ summary: '创建新用户' })
@ApiResponse({ status: 201, description: '用户创建成功' })
async create(@Body() createUserDto: CreateUserDto) {
  return this.userService.create(createUserDto);
}

@Get('status/:status')
@ApiOperation({ summary: '根据状态获取用户列表' })
async findByStatus(@Param('status') status: string) {
  return this.userService.findByStatus(status);
}
```

6. 应用程序配置

步骤 10: 主应用配置 (main.ts)

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // 全局验证管道
  app.useGlobalPipes(new ValidationPipe({
    whitelist: true,
```

```

        transform: true,
        forbidNonWhitelisted: true,
    }));

    // 启用 CORS
    app.enableCors({
        origin: '*',
        methods: 'GET, HEAD, PUT, PATCH, POST, DELETE',
    });

    // 配置 Swagger
    const config = new DocumentBuilder()
        .setTitle('NodeJS Serverless API')
        .setDescription('NodeJS 及 Serverless 应用开发实验项目 API 文档')
        .setVersion('1.0')
        .build();

    const document = SwaggerModule.createDocument(app, config);
    SwaggerModule.setup('api', app, document);

    await app.listen(3000);
}

```

步骤 11: 模块配置 (app.module.ts)

```

@Module({
    imports: [
        ConfigModule.forRoot({
            isGlobal: true,
            envFilePath: '.env',
        }),
        MongooseModule.forRoot(process.env.MONGODB_URI
'mongodb://localhost:27017/nodejs-experiment'),
        UserModule,
        ProductModule,
        InitModule,
    ],
})
export class AppModule {}

```

7. 数据库初始化

步骤 12: 初始化服务 (database-init.service.ts)

```

async initDatabase() {
    console.log('开始初始化数据库...');

    const userCount = await this.userModel.countDocuments().exec();
}

```

```

const productCount = await this.productModel.countDocuments().exec();

if (userCount > 0 && productCount > 0) {
  console.log('数据库已有数据，跳过初始化');
  return;
}

// 创建示例数据
if (userCount === 0) {
  await this.createSampleUsers();
}

if (productCount === 0) {
  await this.createSampleProducts();
}
}

```

8. API 测试

步骤 13: HTTP 测试文件 (api-test.http)

创建新用户

POST http://localhost:3000/users

Content-Type: application/json

```

{
  "username": "test_user",
  "email": "test@example.com",
  "password": "password123",
  "age": 25,
  "status": "active",
  "role": "user"
}

```

创建新产品

POST http://localhost:3000/products

Content-Type: application/json

```

{
  "name": "测试产品",
  "description": "这是一个测试产品",
  "price": 99.99,
  "stock": 100,
  "category": "other"
}

```

9. Serverless 部署配置 (optional)

步骤 14: AWS Lambda 配置 (serverless.yml)

```
service: nodejs-serverless-app

provider:
  name: aws
  runtime: nodejs18.x
  region: ap-beijing
  environment:
    NODE_ENV: production
    MONGODB_URI: ${ssm:/nodejs-app/mongodb-uri~true}

functions:
  api:
    handler: dist/main.handler
    events:
      - httpApi:
          path: /{proxy+}
          method: ANY
```

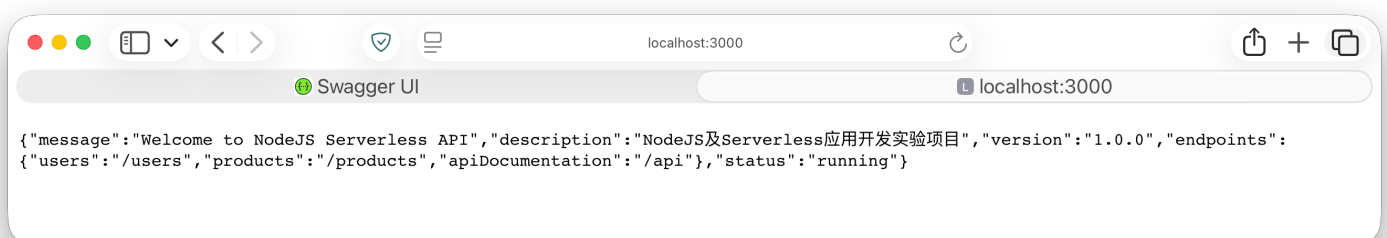
步骤 15: 腾讯云函数配置 (tencent-serverless.yml)

```
component: scf
name: nodejs-serverless-app

inputs:
  name: nodejs-serverless-app
  src: ./dist
  runtime: Nodejs16.15
  region: ap-beijing
  handler: main.handler
  memorySize: 256
  timeout: 60
```

实验效果:

使用 `npm run start:dev` 启动项目:



Swagger
Supported by SMARTBEAR

NodeJS Serverless API1.0OAS 3.0

NodeJS及Serverless应用开发实验项目API文档

users

POST/users创建新用户

GET/users获取所有用户列表

GET/users/{id}根据ID获取用户

PATCH/users/{id}更新用户信息

DELETE/users/{id}删除用户

GET/users/username/{username}根据用户名获取用户

GET/users/status/{status}根据状态获取用户列表

GET/users/role/{role}根据角色获取用户列表

products

localhost:3000/api/#

localhost:3000/api#/users/UserController_findAll

Curl

curl -X 'GET' \n'http://localhost:3000/users' \n-H 'accept: */*'

Request URL

http://localhost:3000/users

Server response

CodeDetails

200

Response body

[
 {
 "_id": "6938c24b05f62d9396c18d3c",
 "username": "admin",
 "email": "admin@example.com",
 "password": "admin123456",
 "age": 30,
 "status": "active",
 "role": "admin",
 "__v": 0,
 "createdAt": "2025-12-10T00:43:55.561Z",
 "updatedAt": "2025-12-10T00:43:55.561Z"
 },
 {
 "_id": "6938c24b05f62d9396c18d3d",
 "username": "john_doe",
 "email": "john@example.com",
 "password": "john123456",
 "age": 25,
 "status": "active",
 "role": "user",
 "__v": 0,
 "createdAt": "2025-12-10T00:43:55.562Z",
 "updatedAt": "2025-12-10T00:43:55.562Z"
 }
]

Download

Response headers

access-control-allow-origin: *
connection: keep-alive
content-length: 945
content-type: application/json; charset=utf-8
date: Wed, 10 Dec 2025 00:54:48 GMT
etag: W/"3b1-wj6W12jRdc3vUS9hdOkL2xQ/7lw"
keep-alive: timeout=5
x-powered-by: Express

products

POST

/products

创建新产品

GET

/products

获取所有产品列表

GET

/products/{id}

根据ID获取产品

PATCH

/products/{id}

更新产品信息

DELETE

/products/{id}

删除产品

GET

/products/search/{name}

根据名称搜索产品

GET

/products/category/{category}

根据分类获取产品列表

GET

/products/status/{status}

根据状态获取产品列表

GET

/products/price-range

根据价格范围获取产品列表

GET

/products/low-stock/list

获取低库存产品（库存小于10）

GET

/products/popular/list

获取热门产品（按销售数量排序）

PATCH

/products/{id}/increase-stock

增加产品库存

PATCH

/products/{id}/decrease-stock

减少产品库存

App

Curl

```
curl -X 'GET' \
'http://localhost:3000/products' \
-H 'accept: */*'
```

Request URL

```
http://localhost:3000/products
```

Server response

Code

Details

200

Response body

```
[
  {
    "_id": "6938c24b05f62d9396c18d41",
    "name": "iPhone 15 Pro",
    "description": "苹果最新款智能手机，配备A17 Pro芯片",
    "price": 7999,
    "stock": 50,
    "category": "electronics",
    "imageUrl": "https://example.com/iphone15.jpg",
    "tags": [
      "手机",
      "苹果",
      "5G"
    ],
    "status": "available",
    "rating": 4.8,
    "salesCount": 120,
    "__v": 0,
    "createdAt": "2025-12-10T00:43:55.567Z",
    "updatedAt": "2025-12-10T00:43:55.567Z"
  },
  {
    "_id": "6938c24b05f62d9396c18d42",
    "name": "MacBook Pro 14寸",
    "description": "苹果专业级笔记本电脑，M2 Pro芯片"
  }
]
```

Download

Response headers

```
access-control-allow-origin: *
connection: keep-alive
content-length: 1926
content-type: application/json; charset=utf-8
date: Wed, 10 Dec 2025 00:55:07 GMT
etag: W/"786-Ys73ggFEuK8j76j0Q8edtLqYNHM"
keep-alive: timeout=5
x-powered-by: Express
```